

Task on ASG

1. Create one VPC in N. Virginia region.

Objective:

To create a VPC in the N. Virginia (us-east-1) region for deploying AWS resources.

Steps:

Open AWS Console and select N. Virginia (us-east-1).

Go to VPC → Your VPCs.

Click Create VPC.

Select VPC only.

Enter the following details:

Name: DevOps-ASG-VPC

IPv4 CIDR: 10.0.0.0/16

Click Create VPC.

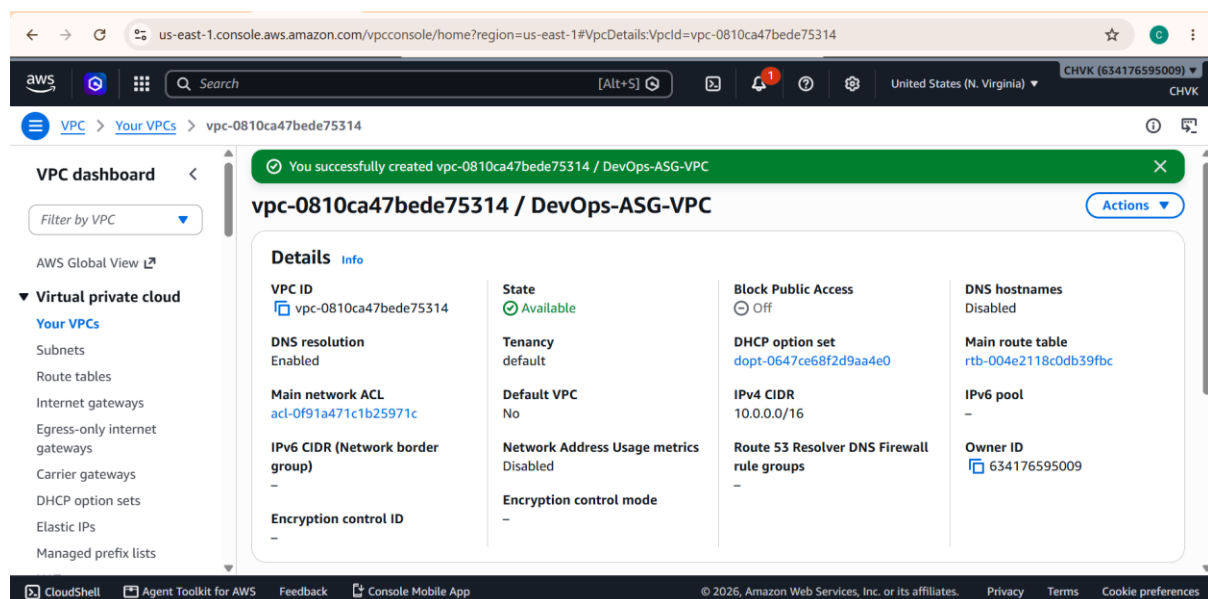
Verify that the VPC status is Available.

Result:

The VPC was successfully created in the N. Virginia region with CIDR 10.0.0.0/16.

Evidence:

Screenshot showing the created VPC name, CIDR block, region, and Available status.



2. Create two subnets: one public subnet and one private subnet.

Objective:

To create one public subnet and one private subnet inside the VPC.

Process:

Open AWS Console → VPC → Subnets.

Click Create subnet.

Select VPC: DevOps-ASG-VPC.

Create the public subnet:

Name: Public-Subnet

AZ: us-east-1a

CIDR: 10.0.1.0/24

Create the private subnet:

Name: Private-Subnet

AZ: us-east-1a

CIDR: 10.0.2.0/24

Enable Auto-assign Public IPv4 for the public subnet.

Click Create subnet.

Result:

Public and private subnets were successfully created inside the VPC.

Evidence:

Screenshot showing both created subnets.

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#subnets:subnetId=subnet-0cbb02be4ec277ce5

aws [Search] [Alt+S] United States (N. Virginia) CHVK (634176595009) CHVK

VPC > Subnets

VPC dashboard < Filter by VPC

AWS Global View

▼ Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

Carrier gateways

DHCP option sets

Elastic IPs

Managed prefix lists

You have successfully created 1 subnet: subnet-0cbb02be4ec277ce5

Subnets (1) Info Last updated less than a minute ago Actions Create subnet

Find subnets by attribute or tag

Subnet ID : subnet-0cbb02be4ec277ce5 Clear filters

☐	Name	Subnet ID	State	VPC
☐	ASG-pub-subnet	subnet-0cbb02be4ec277ce5	Available	vpc-0810ca47bede75314 Dev...

Select a subnet

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#subnets:subnetId=subnet-08f68af3af48f8835

aws [Search] [Alt+S] United States (N. Virginia) CHVK (634176595009) CHVK

VPC > Subnets

VPC dashboard < Filter by VPC

AWS Global View

▼ Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

Carrier gateways

DHCP option sets

Elastic IPs

Managed prefix lists

You have successfully created 1 subnet: subnet-08f68af3af48f8835

Subnets (1) Info Last updated less than a minute ago Actions Create subnet

Find subnets by attribute or tag

Subnet ID : subnet-08f68af3af48f8835 Clear filters

☐	Name	Subnet ID	State	VPC
☐	SAG-Privt-subnet	subnet-08f68af3af48f8835	Available	vpc-0810ca47bede75314 Dev...

Select a subnet

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#SubnetDetails:subnetId=subnet-0cbb02be4ec277ce5

aws [Search] [Alt+S] United States (N. Virginia) CHVK (634176595009) CHVK

VPC > Subnets > subnet-0cbb02be4ec277ce5

VPC dashboard < Filter by VPC

AWS Global View

▼ Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

Carrier gateways

DHCP option sets

Elastic IPs

Managed prefix lists

You have successfully changed subnet settings:

- Enable auto-assign public IPv4 address

subnet-0cbb02be4ec277ce5 / ASG-pub-subnet Actions

Details

Subnet ID subnet-0cbb02be4ec277ce5	Subnet ARN arn:aws:ec2:us-east-1:634176595009:subnet/subnet-0cbb02be4ec277ce5	State Available	Block Public Access Off
IPv4 CIDR 10.0.1.0/24	Available IPv4 addresses 251	IPv6 CIDR -	IPv6 CIDR association ID -
Availability Zone us-east-1a	Network border group us-east-1	VPC vpc-0810ca47bede75314 DevOps-ASG-VPC	Route table -
Network ACL -	Default subnet No	Auto-assign public IPv4 address Yes	Auto-assign IPv6 address No
Auto-assign customer-owned IPv4 address No	Customer-owned IPv4 pool -	Outpost ID -	IPv4 CIDR reservations -
			Resource name DNS A record -

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

3. Attach an IGW to the VPC.

Objective:

To create an Internet Gateway and attach it to the VPC for internet connectivity.

Process:

Open VPC → Internet Gateways.

Click Create Internet Gateway.

Enter the name DevOps-IGW.

Click Create Internet Gateway.

Select DevOps-IGW.

Go to Actions → Attach to VPC.

Select DevOps-ASG-VPC.

Click Attach Internet Gateway.

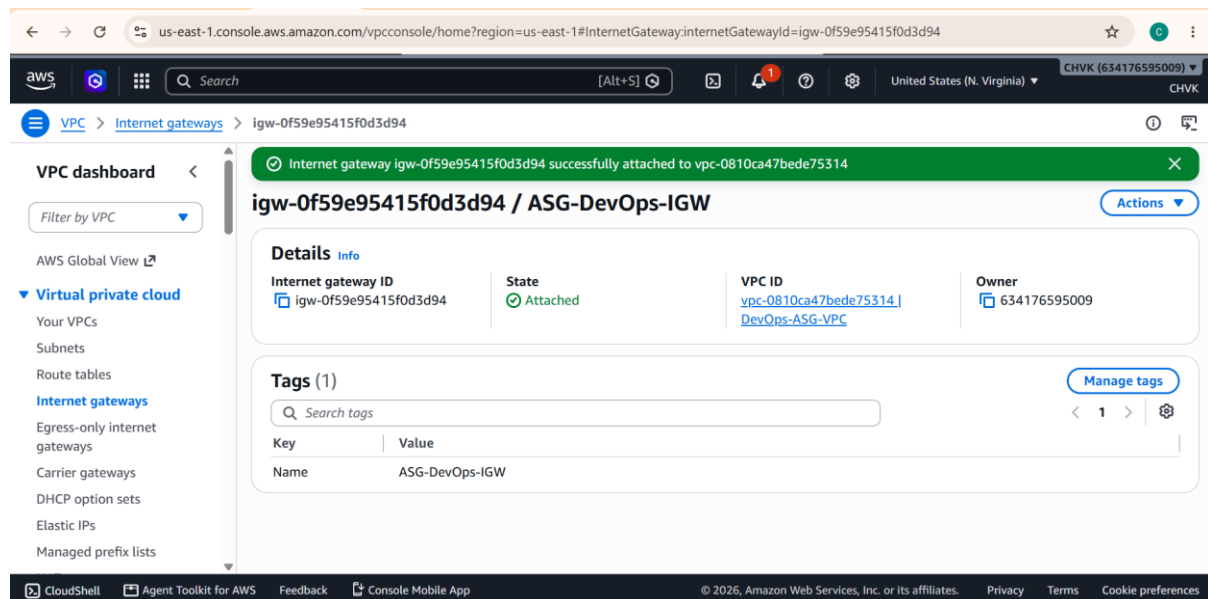
Verify that the state is Attached.

Result:

The Internet Gateway was successfully created and attached to the VPC.

Evidence:

Screenshot showing DevOps-IGW attached to DevOps-ASG-VPC.



4. Create one public route table (RT) and one private route table.

Objective:

To create separate route tables for the public and private subnets.

Process:

Open VPC → Route Tables.

Click Create route table.

Create Public-RT and select DevOps-ASG-VPC.

Add route 0.0.0.0/0 → Internet Gateway (DevOps-IGW).

Associate Public-Subnet with Public-RT.

Create another route table named Private-RT.

Select DevOps-ASG-VPC.

Associate Private-Subnet with Private-RT.

Keep only the default local route in the private route table for now. NAT Gateway route will be added in the next task.

Result:

Public and private route tables were successfully created and associated with their respective subnets.

Evidence:

Screenshots showing the public route, private route table, and subnet associations.

The first screenshot shows the AWS Management Console for the route table `rtb-0687f717ebe0f8296`. A green notification bar at the top states: "Route table rtb-0687f717ebe0f8296 / ASG-Puvlic-RT was created successfully." The route table details show it is associated with VPC `vpc-0810ca47bede75314` (DevOps-ASG-VPC) and has no explicit subnet associations or edge associations. The "Routes" tab shows one route with destination `0.0.0.0/0` and target `local`, with a status of "Active".

The second screenshot shows the same route table after an update. A green notification bar states: "Updated routes for rtb-0687f717ebe0f8296 / ASG-Puvlic-RT successfully". The "Routes" tab now shows two routes:

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-0f59e95415f0d3...	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

The top screenshot shows the AWS Management Console for the VPC dashboard, specifically the 'Route tables' section. A green notification bar at the top states: 'You have successfully updated subnet associations for rtb-0687f717ebe0f8296 / ASG-Public-RT.' The main content area displays details for the route table 'rtb-0687f717ebe0f8296', including its VPC 'vpc-0810ca47bede75314 | DevOps-ASG-VPC' and Owner ID '634176595009'. The 'Subnet associations' tab is selected, showing 'Explicit subnet associations (1)' with a table containing one entry: 'ASG-pub-subnet' with Subnet ID 'subnet-0cbb02be4ec277ce5' and IPv4 CIDR '10.0.1.0/24'. Below this, it lists 'Subnets without explicit associations (1)'.

The bottom screenshot shows the same console for a different route table, 'rtb-0857d53c19115a66a'. A green notification bar states: 'You have successfully updated subnet associations for rtb-0857d53c19115a66a / ASG-privt-RT.' The details for this route table show VPC 'vpc-0810ca47bede75314 | DevOps-ASG-VPC' and Owner ID '634176595009'. The 'Subnet associations' tab is selected, showing 'Explicit subnet associations (1)' with a table containing one entry: 'SAG-Privt-subnet' with Subnet ID 'subnet-08f68af3af48f8835' and IPv4 CIDR '10.0.2.0/24'.

5. Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.

Objective:

To create a NAT Gateway in the public subnet and provide internet access to the private subnet.

Process:

Open VPC → NAT Gateways.

Click Create NAT Gateway.

Enter the name DevOps-NAT-GW.

Select Public-Subnet.

Select Public connectivity type.

Allocate a new Elastic IP.

Click Create NAT Gateway.

Wait until the NAT Gateway status becomes Available.

Go to Route Tables → Private-RT.

Edit routes and add 0.0.0.0/0 with target DevOps-NAT-GW.

Save the route.

Result:

The NAT Gateway was successfully created in the public subnet and connected to the private route table.

Evidence:

Screenshot showing NAT Gateway status as Available and the private route table route pointing to the NAT Gateway.

The top screenshot displays the AWS Management Console page for the NAT Gateway 'nat-1c722c9840b9fe542 / ASG-DevOps-NAT-GW'. The 'Details' section shows the NAT gateway ID, ARN, VPC, availability mode (Regional), connectivity type (Public), and state (Available). The 'Associated IP addresses' section shows one IP address associated with the gateway.

The bottom screenshot displays the AWS Management Console page for the Route Table 'rtb-0857d53c19115a66a'. A green notification banner at the top states 'Updated routes for rtb-0857d53c19115a66a / ASG-privt-RT successfully'. The 'Routes' section shows two routes: one for destination 0.0.0.0/0 with target 'nat-1c722c9840b9fe5...' and status 'Active', and another for destination 10.0.0.0/16 with target 'local' and status 'Active'.

6. Create two instances, one in the public subnet and one in the private subnet.

Objective:

To create two EC2 instances, one in the public subnet and one in the private subnet.

Process:

Open EC2 → Instances → Launch Instance.

Create the first instance:

Name: Apache-Public-EC2

AMI: Amazon Linux 2023

VPC: DevOps-ASG-VPC

Subnet: Public-Subnet

Public IP: Enabled

Allow SSH (22) and HTTP (80) in the security group.

Launch the instance.

Create the second instance:

Name: Apache-Private-EC2

AMI: Amazon Linux 2023

VPC: DevOps-ASG-VPC

Subnet: Private-Subnet

Public IP: Disabled

Allow the required HTTP access in the security group.

Launch the instance.

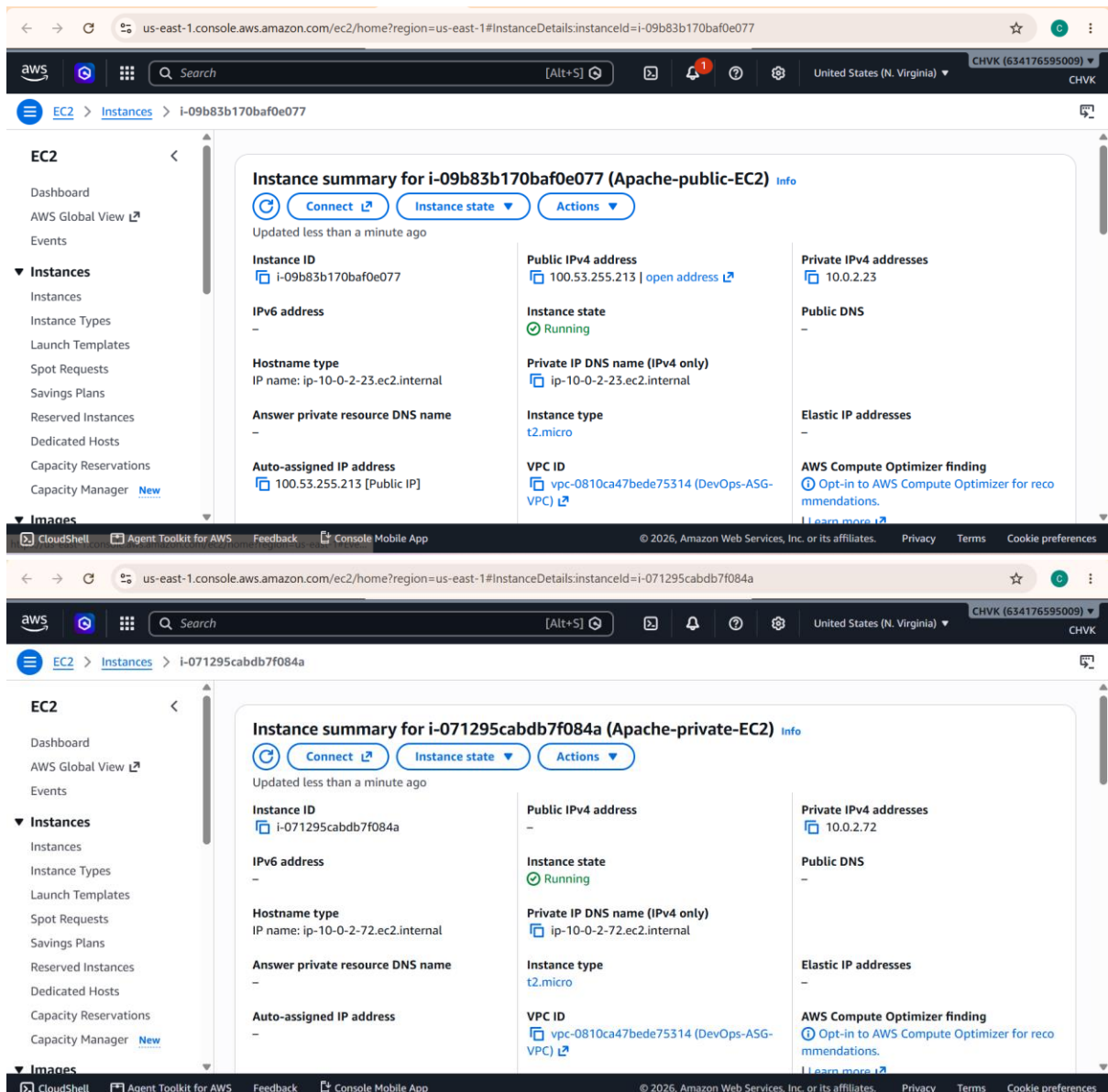
Verify that both instances are in the Running state.

Result:

Two EC2 instances were successfully created, one in the public subnet and one in the private subnet.

Evidence:

Screenshot showing both EC2 instances, their respective subnets, and running status.



7. Deploy Apache server on both EC2 instances with a sample index.html file.

Objective:

To install and configure Apache web server on both public and private EC2 instances with a sample index.html page.

Process:

Connect to the Public EC2 instance.

Install Apache using: `sudo dnf install httpd -y`

Start Apache: `sudo systemctl start httpd`

Enable Apache: `sudo systemctl enable httpd`

Create a sample index.html file in `/var/www/html/`.

Verify Apache using: `sudo systemctl status httpd`

Test the webpage using: `curl http://localhost`

Repeat the same steps on the Private EC2 instance.

Verify that Apache is running on both instances.

Result:

Apache web server was successfully installed and configured on both EC2 instances with a sample index.html page.

Evidence:

Screenshots showing Apache service status and index.html output from both EC2 instances.

```
[ec2-user@ip-10-0-1-233 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-10 09:49:08 UTC; 32s ago
     Docs: man:httpd.service(8)
  Main PID: 26689 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
     Tasks: 177 (limit: 1113)
    Memory: 13.3M
       CPU: 67ms
    CGroup: /system.slice/httpd.service
            └─26689 /usr/sbin/httpd -DFOREGROUND
              └─26690 /usr/sbin/httpd -DFOREGROUND
                └─26691 /usr/sbin/httpd -DFOREGROUND
                  └─26692 /usr/sbin/httpd -DFOREGROUND
                    └─26712 /usr/sbin/httpd -DFOREGROUND

Sep 10 09:49:08 ip-10-0-1-233.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Sep 10 09:49:08 ip-10-0-1-233.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Sep 10 09:49:08 ip-10-0-1-233.ec2.internal httpd[26689]: Server configured, listening on: port 80
[ec2-user@ip-10-0-1-233 ~]$ sudo vi /var/www/html/index.html
[ec2-user@ip-10-0-1-233 ~]$ curl http://localhost
<h1>HELLO FROM KARTHIK</h1>
<p>Apache server is Running</p>

[ec2-user@ip-10-0-1-233 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-10 09:49:08 UTC; 3min 7s ago
     Docs: man:httpd.service(8)
  Main PID: 26689 (httpd)
    Status: "Total requests: 1; Idle/Busy workers 100/0; Requests/sec: 0.00559; Bytes served/sec: 2 B/sec"
     Tasks: 177 (limit: 1113)
    Memory: 13.4M
       CPU: 158ms
    CGroup: /system.slice/httpd.service
            └─26689 /usr/sbin/httpd -DFOREGROUND
              └─26690 /usr/sbin/httpd -DFOREGROUND
                └─26691 /usr/sbin/httpd -DFOREGROUND
                  └─26692 /usr/sbin/httpd -DFOREGROUND
                    └─26712 /usr/sbin/httpd -DFOREGROUND

Sep 10 09:49:08 ip-10-0-1-233.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Sep 10 09:49:08 ip-10-0-1-233.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Sep 10 09:49:08 ip-10-0-1-233.ec2.internal httpd[26689]: Server configured, listening on: port 80
[ec2-user@ip-10-0-1-233 ~]$ sudo vi /var/www/html/index.html
[ec2-user@ip-10-0-1-233 ~]$ curl http://localhost
<h1>HELLO FROM PRIVATE EC2</h1>
<p>Apache server is Running</p>
```

8. Create one application load balancer and attach it to both EC2 instances.

Objective:

To create an Application Load Balancer (ALB) and distribute HTTP traffic between the Public and Private Apache EC2 instances.

Process:

Created a Target Group named Apache-TG.

Configured the target group with HTTP protocol on port 80.

Registered both Apache-Public-EC2 and Apache-private-EC2 instances as targets.

Created an internet-facing Application Load Balancer named Devops-ALB.

Selected the Public subnets in us-east-1a and us-east-1b for the ALB.

Configured the HTTP listener on port 80 and forwarded requests to Apache-TG.

Verified the ALB using its DNS name: devops-alb-307014288.us-east-1.elb.amazonaws.com

Opened the ALB DNS name in a browser and successfully received the Apache webpage from the Private EC2 instance.

Result:

The Application Load Balancer was successfully created and connected to both Apache EC2 instances. The ALB DNS name successfully displayed the Apache webpage.

Evidence:

Target Group showing both EC2 instances registered.

ALB details showing Devops-ALB.

ALB DNS name.

Browser showing HELLO FROM PRIVATE EC2 – Apache server is Running.

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#TargetGroup:targetGroupArn=arn:aws:elasticloadbalancing:us-east-1:634176595009:targetgr...

Search [Alt+S] Ask Amazon Q United States (N. Virginia) CHVK (634176595009) CHVK

EC2 > Target groups > Apache-TG

Network & Security

- Security Groups
- Elastic IPs
- Placement Groups
- Key Pairs
- Network Interfaces

Load Balancing

- Load Balancers
- Target Groups
- Trust Stores

Auto Scaling

- Auto Scaling Groups

Settings

Successfully created the target group: **Apache-TG**. Anomaly detection is automatically applied to all registered targets. Results can be viewed in the **Targets** tab.

Apache-TG Actions

Details

arn:aws:elasticloadbalancing:us-east-1:634176595009:targetgroup/Apache-TG/bb8c38b836c8689c

Target type Instance	Protocol : Port HTTP: 80	Protocol version HTTP1	VPC vpc-0810ca47bede75314
IP address type IPv4	Load balancer None associated		

2 Total targets	0 Healthy	0 Unhealthy	2 Unused	0 Initial	0 Draining
	0 Anomalous				

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LoadBalancer:loadBalancerArn=arn:aws:elasticloadbalancing:us-east-1:634176595009:loadbal...

Search [Alt+S] United States (N. Virginia) CHVK (634176595009) CHVK

EC2 > Load balancers > Devops-ALB

EC2

- Dashboard
- AWS Global View
- Events

Instances

- Instances
- Instance Types
- Launch Templates
- Spot Requests
- Savings Plans
- Reserved Instances
- Dedicated Hosts
- Capacity Reservations
- Capacity Manager [New](#)

Images

Successfully created load balancer: **Devops-ALB**. It might take a few minutes for your load balancer to fully set up and route traffic. Targets will also take a few minutes to complete the registration process and pass initial health checks.

Introducing new Capacity Utilization metric for ALB. Monitor how much of your load balancer's available capacity was used to meet traffic demand. Visit the **Capacity** or **Monitoring** tabs to view utilization data.

Devops-ALB Actions

Details

Load balancer type Application	Status Active	VPC vpc-0810ca47bede75314	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone Z35SXDOTRQ7X7K	Availability Zones subnet-0a573a8811abff7e7 us-east-1b (use1-az2)	Date created September 10, 2026, 15:40 (UTC+05:30)

CloudShell Agent Toolkit for AWS Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Not secure devops-alb-307014288.us-east-1.elb.amazonaws.com

HELLO FROM PRIVATE EC2

Apache server is Running

9. Store application load balancer logs in S3.

Objective:

To configure the Application Load Balancer to store access logs in an Amazon S3 bucket.

Process:

Created an S3 bucket named devops-alb-logs-2026-001.

Opened EC2 → Load Balancers → Devops-ALB.

Opened Attributes → Edit load balancer attributes.

Enabled Access logs.

Configured the S3 destination as s3://devops-alb-logs-2026-001.

Updated the S3 bucket permissions to allow ALB log delivery.

Saved the load balancer attributes successfully.

Result:

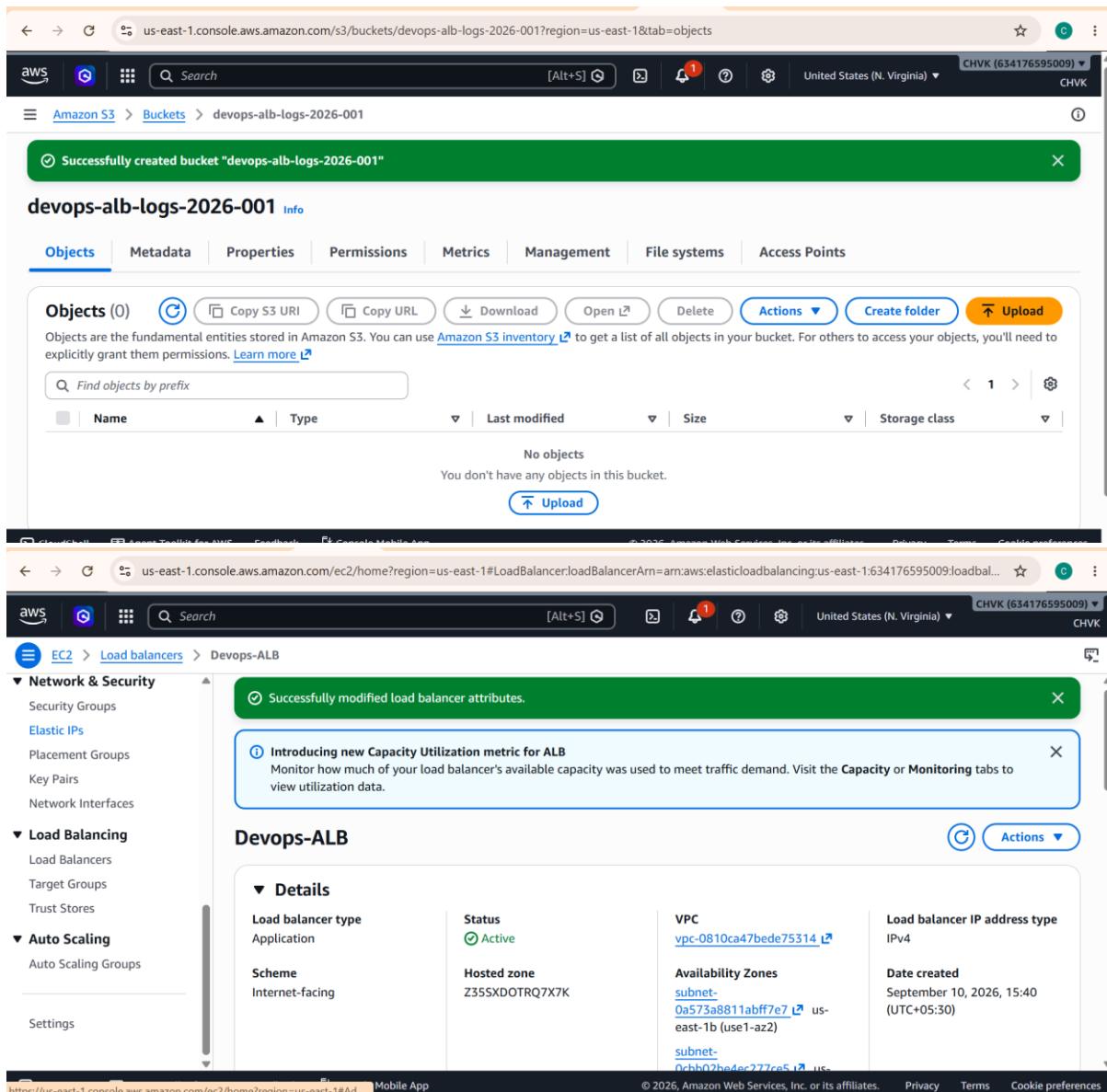
ALB access logging was successfully enabled, and the ALB is configured to deliver access logs to the S3 bucket.

Evidence:

S3 bucket devops-alb-logs-2026-001

ALB Access Logs configuration

“Successfully modified load balancer attributes” message



10. Store the VPC flow logs in a CloudWatch log group.

Objective:

To enable VPC Flow Logs for the VPC and send the network traffic logs to Amazon CloudWatch.

Process:

Selected the DevOps-ASG-VPC VPC.

Opened Actions → Create flow log.

Created the flow log with the name DevOps-VPC-Flow-Logs.

Selected All traffic for the flow log filter.

Set the maximum aggregation interval to 1 minute.

Selected Send to CloudWatch Logs as the destination.

Configured the CloudWatch Log Group as /devops/vpc-flow-logs.

Selected Create and use a new service role to provide the required permissions.

Created the VPC Flow Log successfully.

Result:

VPC Flow Logs were successfully created for DevOps-ASG-VPC and configured to send network traffic logs to the CloudWatch Log Group.

Evidence:

VPC Flow Log DevOps-VPC-Flow-Logs

CloudWatch Log Group /devops/vpc-flow-logs

Successful Flow Log creation

us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#logsV2:log-groups

CloudWatch > Log management

Log group "/devops/vpc-flow-logs" has been created.

Log groups (3)

By default, we only load up to 10,000 log groups.

Filter log groups or try pattern search

Exact match

Log group	Log class	Anomaly d...	De...	Bea...	D...
/devops/vpc-flow-logs	Standard	Configure	Off	Off	-
/aws/cloudtrail/devops	Standard	Configure	Off	Off	-

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#FlowLogDetails:flowLogId=fl-0fb3563541b783dc5

VPC > Your VPCs > fl-0fb3563541b783dc5

Successfully created flow log for the following resource:

vpc-0810ca47bede75314

fl-0fb3563541b783dc5 / DevOps-VPC-Flow-Logs

Details

Flow Log ID fl-0fb3563541b783dc5	Destination Type cloud-watch-logs	Traffic Type All	File Format -
Name DevOps-VPC-Flow-Logs	Destination Name /devops/vpc-flow-logs	Max Aggregation Interval 1 minute	Hive Compatible Partitions -
State Active	IAM Role arn:aws:iam::634176595009:role/service-role/VPCFlowLogs-Cloudwatch-1789038932216	Log Format Default	Partition Logs -
Creation Time Thursday, September 10, 2026	Cross Account IAM Role -	Tag field specifications -	

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#VpcDetails:vpclId=vpc-0810ca47bede75314

VPC > Your VPCs > vpc-0810ca47bede75314

Main network ACL
acl-0f91a471c1b25971c

IPv6 CIDR (Network border group)
-

Encryption control ID
-

Default VPC
No

Network Address Usage metrics
Disabled

Encryption control mode
-

IPv4 CIDR
10.0.0.0/16

Route 53 Resolver DNS Firewall rule groups
-

IPv6 pool
-

Owner ID
634176595009

Resource map CIDRs Flow logs Tags Related resources Integrations

Flow logs (1)

Search

Maximum aggregation interval	Creation time	Status	Log line form
1 minute	Thursday, September 10, 2026 at 16:48...	Active	Default

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

11. Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.

Objective:

To create a CloudWatch monitoring dashboard for monitoring EC2 CPU utilization and Apache web server activity.

Process:

Created a CloudWatch dashboard named DevOps-Monitoring-Dashboard.

Added the CPUUtilization metric for the EC2 instances.

Saved the CPU utilization graph to the dashboard.

Installed the Amazon CloudWatch Agent on the Public EC2 instance.

Created a CloudWatch Agent configuration to collect Apache HTTP server logs.

Configured Apache access and error logs to be sent to the /devops/apache CloudWatch Log Group.

Started the Amazon CloudWatch Agent.

Verified that the CloudWatch Agent status is Active (running).

Result:

The CloudWatch monitoring dashboard was successfully created, and EC2 CPU utilization and Apache server monitoring were configured successfully.

Evidence:

DevOps-Monitoring-Dashboard

CPUUtilization graph

CloudWatch Agent configuration

Amazon CloudWatch Agent status showing Active (running)

/devops/apache CloudWatch Log Group

The screenshot displays the AWS CloudWatch DevOps-Monitoring-Dashboard. The top section shows CPU utilization for two instances: 1.24% and 1.36%. Below this, a CloudShell terminal window shows the status of the Amazon CloudWatch agent. The agent is active and running, with a PID of 32797. The terminal output includes the following details:

```

[ec2-user@ip-10-0-1-233 ~]$ sudo systemctl status amazon-cloudwatch-agent
● amazon-cloudwatch-agent.service - Amazon CloudWatch Agent
   Loaded: loaded (/etc/systemd/system/amazon-cloudwatch-agent.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-10 11:35:30 UTC; 19s ago
   Main PID: 32797 (amazon-cloudwat)
     Tasks: 6 (limit: 1113)
    Memory: 52.7M
       CPU: 264ms
   CGroup: /system.slice/amazon-cloudwatch-agent.service
           └─32797 /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent --config /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml -envconfig /opt/

```

The bottom section of the screenshot shows the CloudWatch Log Management console. The 'Log streams' tab is selected, displaying a list of log streams for the 'i-00e4167e2858b5a9e/apache-error' and 'i-00e4167e2858b5a9e/apache-access' log groups. The log streams are filtered by 'Last event time' and show the most recent events.

12. If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

Objective

To configure Auto Scaling so that a new EC2 instance can be launched automatically when CPU utilization increases above the configured threshold.

Process

Created a Launch Template named Devops-asg-launch-template.

Configured the Launch Template with:

Amazon Linux 2023

Instance type: t2.micro

Key pair: ec2-task-key

Security Group: devops-asg-sg-new

Added User Data to automatically install and start Apache Web Server.

Created an Auto Scaling Group named Devops-asg.

Selected VPC DevOps-ASG-VPC and the configured public subnets.

Attached the Auto Scaling Group to the existing target group Apache-TG.

Configured:

Desired capacity: 1

Minimum capacity: 1

Maximum capacity: 2

Configured the scaling policy with Average CPU Utilization = 70%.

Verified that the Auto Scaling Group launched an EC2 instance successfully.

Verified that the instance health showed 1/1 Healthy.

Result

The Auto Scaling Group Devops-asg was successfully created and configured with a minimum of 1 and maximum of 2 instances. The launched instance is healthy and connected to the configured target group.

✓ Success

Successfully created [Devops-asg-launch-template\(lt-03dbaac720939a87b\)](#).

▶ Actions log

Next Steps

Launch an instance

With On-Demand Instances, you pay for compute capacity by the second (for Linux, with a minimum of 60 seconds) or by the hour (for all other operating systems) with no long-term commitments or upfront payments. Launch an On-Demand Instance from your launch template.

[Launch instance from this template](#)

Create an Auto Scaling group from your template

Amazon EC2 Auto Scaling helps you maintain application availability and allows you to scale your Amazon EC2 capacity up or down automatically according to conditions you define. You can use Auto Scaling to help ensure that you are running your desired number of Amazon EC2 instances during demand spikes to maintain performance and decrease capacity during lulls to reduce costs.

[Create Auto Scaling group](#)

Auto Scaling groups (1) [Info](#)

Last updated less than a minute ago

🔄

Launch configurations

Launch templates [↗](#)

Actions ▾

Create Auto Scaling group

🔍 Search your Auto Scaling groups

< 1 > ⚙️

	Launch template/configuration ↗	Instances ▾	Instance health - new ▾	Desired capacity ▾	Min ▾	Max ▾	Availability Zone
↓ capacity	Devops-asg-launch-template Version De	1	✓ 1/1 Healthy	1	1	2	2 Availability Zon

0 Auto Scaling groups selected

⤴